Republic of the Philippines
CARAGA STATE UNIVERSITY
CABADBARAN CAMPUS
T. Curato St., Cabadbaran City, Agusan del Norte
Competence Service Uprightness

COLLEGE OF ENGINEERING AND INFORMATION TECHNOLOGY

LABORATORY MANUAL ACTIVITY

Selecting and Planning the Software Development Methodology for Your Own Project

Software Development Methodologies

Course	IT 110 – Web Systems and Technologies
Activity	Laboratory Activity No. 4
Output	Development methodology plan, selection worksheets, milestone map, online justification, and group presentation
Maximum Score	80 points

Student / Project Information

Block: BSIT - 3A1 IBPH3	Date: September 17, 2026
System Title: Humanitarian Aid Distribution Management System with Machine Learning-Based Household Vulnerability Classification	Instructor: Jean Y. Villasin, MIT
Members: Steven P. Payapaya **Kyle M. Borja**	

1. Activity Alignment and Purpose

Topic Learning Outcome: Compare common software development methodologies and select an appropriate approach for planning and managing the semester web-system project.

Course Outcome Alignment: Supports project planning and management by selecting and applying an appropriate development approach to the semester web-system project.

Syllabus Assessment: Development methodology plan; online methodology-selection justification.

Purpose of the Activity
Activities 2 and 3 established the architecture and technology stack of your semester project. In Activity 4, your team will decide how the project will be planned, developed, reviewed, and delivered. You will compare common methodologies, analyze project conditions, select a defensible primary approach, and map it to SDLC phases, iterations or sprints, and semester milestones. The final plan should become the team's working guide for managing development during the remaining laboratory activities.

Important distinction: SDLC phases describe the major categories of development work, while a methodology or development approach organizes how those phases are planned and repeated. Agile approaches do not remove requirements, design, testing, or deployment; they arrange work in adaptive cycles. DevOps and CI/CD are complementary delivery practices and do not replace the selected development methodology.

2. Intended Learning Objectives

- Explain the SDLC phases and the purpose of using a software development methodology.
- Distinguish predictive, iterative, incremental, adaptive, and hybrid development approaches.
- Compare Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, and Rapid Application Development (RAD).
- Compare Agile approaches including Scrum, Kanban, and Extreme Programming (XP).
- Explain how DevOps and CI/CD complement development and delivery without replacing the project methodology.
- Evaluate requirement stability, risk, scope, schedule, team capability, and stakeholder involvement when selecting a methodology.
- Map the selected methodology to SDLC phases, iterations, sprints, project milestones, reviews, and semester deliverables.

3. Activity Specifications

Activity Item	Requirement
Mode	Existing semester-project group; all members must contribute to the comparison, selection, milestone planning, and oral defense.
Suggested Time	One laboratory session, approximately 3 hours, plus the scheduled group presentation.
Primary Output	One project-specific Development Methodology Plan for the semester web-system project.

Activity Item	Requirement
Supporting Output	Methodology comparison matrix, scenario analysis, selection-criteria worksheet, SDLC mapping, semester milestone plan, DevOps/CI/CD delivery plan, 120-180 word methodology-selection justification, and group presentation.
Recommended Tools	Word/Google Docs, spreadsheet or timeline tool, diagrams.net/Draw.io, Canva or PowerPoint, GitHub Projects/Trello or an equivalent task board, and the assigned methodology case-study reading.
Presentation Format	5-7 minutes per group followed by 2-3 minutes of questions. Every member must explain an assigned selection criterion, project stage/iteration, or milestone.

4. Pre-Laboratory Requirements

- Approved system title and a brief statement of the community or organizational problem being addressed.
- Final or revised web-system architecture diagram from Laboratory Activity No. 2.
- Final or revised technology stack blueprint from Laboratory Activity No. 3.
- List of major system features, primary users, stakeholders, and any required approvals or feedback points.
- Known project constraints such as semester deadlines, technical risks, external services, uncertain requirements, and team limitations.
- The assigned methodology case-study reading and lecture notes on SDLC phases and software development methodologies.
- Access to the group GitHub repository and any current backlog, task board, calendar, or milestone list used by the team.

5. Part A - Development Approach Classification

Before comparing individual methodologies, classify the planning pattern correctly. Use the descriptions below to distinguish how a project responds to requirements, risk, delivery, and change. A specific methodology may combine more than one pattern.

Category	Meaning
SDLC phases	Planning, requirements/analysis, design, development, testing, deployment, and maintenance/review activities that every project must address in some form.
Predictive approach	Plans most scope, sequence, and major deliverables early; works best when requirements are relatively stable and change can be controlled.
Iterative approach	Repeats development cycles to refine the solution, requirements, design, or implementation using learning from earlier cycles.
Incremental approach	Builds and delivers the system in usable pieces or feature increments rather than waiting for one complete final release.
Adaptive / Agile approach	Uses frequent planning, feedback, and change response through short cycles or continuous flow.
Hybrid approach	Combines clearly defined practices from more than one approach because different parts of the project have different needs.
DevOps / CI/CD	Complementary delivery practices that support integration, testing, release, deployment, feedback, and operational reliability; they are not substitutes for the project methodology.

Methodology Comparison Worksheet

Methodology / Approach	Primary Planning Pattern	Strong Fit / Advantage	Limitation / Risk
Waterfall	Predictive	Works well when the requirements are clear and stable.Works well when the requirements are clear and stable.	Changes can be difficult once development has started.
V-Model	Predictive	Gives strong focus on testing and system quality.	It is not very flexible when requirements change.
Prototyping	Iterative	Helps the team understand user needs and improve the interface.	Too many changes to the prototype can take more time.
Iterative	Iterative	Allows the team to improve features based on testing and feedback.	Too many iterations can affect the project schedule.
Incremental	Incremental	Allows the team to complete and test features gradually.	The features must be properly connected and integrated.
Spiral	Risk-driven	Useful for projects with high technical or project risks.	It can be complicated for a small team.
Rapid Application Development (RAD)	Iterative / Incremental	Helps the team create working features in a shorter time.	It needs regular feedback and proper testing.
Scrum	Adaptive / Agile	Supports teamwork, feedback, and changes during development.	It needs good planning and coordination within the team.
Kanban	Adaptive / Agile	Makes tasks and project progress easy to track.	It has less structure for fixed sprint schedules.
Extreme Programming (XP)	Adaptive / Agile	Helps improve software quality through regular testing.	It requires strong teamwork and development discipline.

6. Part B - Project Methodology Selection Criteria

Evaluate the conditions of your own project. Record evidence in the second column, then use the comparison cues to decide whether the condition favors a more predictive or a more iterative/adaptive approach. Your final choice should be based on the combined evidence, not on one criterion alone.

Selection Criterion	Your Project Evidence / Condition	More Predictive Tends to Fit When...	More Iterative / Adaptive Tends to Fit When...	Team Interpretation
Requirement stability	The main system features are already identified, but some details may change during development. Feedback may require adjustments to certain features.	Requirements are known and unlikely to change.	Requirements are expected to evolve through feedback or discovery.	Favors iterative/adaptive.
Technical / project risk	The project includes REST API integration, PostgreSQL, and a Machine Learning Classification Service. These components may have integration issues during development.	Technology and integrations are familiar with few unknowns.	High uncertainty, new technology, risky integrations, or research-heavy work require learning cycles.	Favors an iterative/adaptive approach because technical risks can be identified and addressed during development.
Scope clarity	The main features include household registration, assessment, vulnerability classification, beneficiary management, aid distribution, and reports. These features provide a clear overall project scope.	Major features and boundaries can be defined early.	Feature priorities or details may change as prototypes and feedback are reviewed.	Favors an iterative/adaptive approach because features can be refined as development progresses.
Schedule / deadline	The system is being developed as a semester project with a fixed academic deadline. The team needs to complete the required features within the available time.	Formal phase gates and fixed planned deliverables are practical.	Frequent usable increments or short time-boxes are needed before the final deadline.	Short sprints can make progress easier to track and manage.
Team size and experience	The project team consists of two members who will share development, testing, documentation, and integration tasks. Both members need to coordinate their work closely.	Roles can be specialized and work can follow a planned sequence.	A small cross-functional team can collaborate frequently and adjust tasks quickly.	A simple adaptive process with short sprints is practical for the two-member team.

Stakeholder involvement	The project may receive feedback from the instructor or adviser during development. This feedback can help identify areas that need improvement.	Stakeholders are available mainly at defined approval points.	Stakeholders can provide frequent feedback, review prototypes, or reprioritize work.	Favors regular review and feedback during development.
Need for change and feedback	Some system features, interface details, and ML integration may need adjustments after testing or feedback. These changes may occur while development is already in progress.	Changes are rare, costly, or tightly controlled.	Learning, testing, and user feedback are expected to change the next iteration or work queue.	Strongly supports an adaptive development approach.

Selection rule
Choose a methodology because it fits the project conditions, not because it is popular or familiar. A hybrid approach is acceptable only when the team identifies which practices are combined and where each one applies. DevOps and CI/CD may complement any selected methodology but should not be presented as the methodology itself.

7. Part C - Methodology Scenario-Analysis Workshop

For each scenario, identify a defensible methodology or approach and explain the tradeoff. More than one answer may be acceptable when the reasoning is supported by the project conditions.

Scenario	Key Project Condition	Best-Fit Methodology	Why It Fits	Main Risk / Limitation	Alternative / Hybrid Option
1. Government registration module	Requirements are defined by fixed forms, rules, and approval steps.	Waterfall	The requirements are clear and can be planned before development.	Changes after development may be difficult to manage.	V-Model
2. New citizen-reporting interface	Users are unsure what screen flow will be easiest until they see a working prototype.	Prototyping	Early prototypes allow users to review the interface and provide feedback.	Too many prototype changes may affect the schedule.	Iterative
3. Verification-intensive transaction module	Each development stage requires a corresponding test or verification activity.	V-Model	It provides strong verification and testing throughout development.	It provides strong verification and testing throughout development.	Waterfall
4. Semester project with frequent adviser feedback	A small team must show working features regularly and reprioritize based on reviews.	Scrum	Short sprints allow the team to develop features and receive regular feedback.	Frequent changes may require careful sprint and task management.	Kanban
5. Short deadline for an early working demo	The team needs rapid construction, reusable components, and strong stakeholder availability.	RAD	It supports rapid development of working features for an early demonstration.	It supports rapid development of working features for an early demonstration.	Incremental
6. GIS/ML feature with uncertain integration risk	External APIs, data quality, or model behavior may create technical uncertainty that must be evaluated repeatedly.	Spiral	It allows the team to identify and address technical risks through repeated cycles.	It can be more complex to manage for a small team.	Scrum

Scenario-analysis checklist:

- Use evidence from requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Do not select a methodology only because the team already knows it or because it is currently popular.
- Distinguish the development methodology from tools such as Trello/GitHub Projects and from complementary DevOps/CI/CD practices.
- State at least one tradeoff, limitation, or alternative approach for every scenario.

8. Part D - Select and Map Your Project Methodology

Selected primary methodology: Scrum

Complementary approach / practice(s): Adaptive / Agile

Why this choice fits the project (3-5 sentences):

Scrum fits our project because we are a two-member team developing the system within a semester deadline. The project includes several features such as household assessment, vulnerability classification, aid distribution, and reports that can be developed in smaller increments. The Machine Learning Classification Service may also require testing and adjustments during integration. Scrum allows us to work in short sprints, review our progress, and use instructor or adviser feedback to improve the system.

Project Methodology and SDLC Mapping Worksheet

Map the major SDLC activities to the way your selected methodology will actually organize the semester project. If the methodology repeats phases, show where the repetition occurs. If it uses sprints or continuous flow, identify the planning and review points clearly.

SDLC Phase / Activity	How It Is Handled	Time-Box / Duration	Iteration / Sprint / Stage	Main Team Role(s)	Stakeholder Check	Deliverable / Exit Evidence
Project planning	Define the project scope, priorities, tasks, and overall development plan	Start of project and before each sprint	Project Planning / Sprint Planning	Steven & Kyle	Instructor/adviser review	Project plan and prioritized backlog
Requirements / analysis	Identify and refine system requirements based on the project scope and feedback.	Early project stage and refined each sprint	Initial Analysis + Sprint Backlog	Steven & Kyle	Instructor/adviser feedback	Requirements and updated backlog
Design	Design the system structure, user interface, database mapping, and API flow before implementation.	Design the system structure, user interface, database mapping, and API flow before implementation.	Design Stage / Sprint Planning	Steven & Kyle	Design review	Architecture, UI, and API/database designs
Development / implementation	Develop system features according to the prioritized sprint tasks.	Each sprint	Sprint Development	Steven & Kyle	Feature review	Working system features
Testing / verification	Test completed features and fix errors before considering the work complete.	Throughout each sprint	Throughout each sprint	Steven & Kyle	Review of tested features	Test results and corrected features
Integration / deployment	Integrate Flutter Web, REST API, PostgreSQL, and ML service and prepare the working system for deployment.	During integration and final stages	Integration / Deployment Stage	Steven & Kyle	Integration review	Integrated and deployable system
Review / feedback	Review completed work, collect feedback, and identify improvements for the next sprint.	End of each sprint	Sprint Review	Steven & Kyle	Instructor/adviser feedback	Review results and feedback records
Maintenance / improvement	Fix remaining issues and improve features based on testing and review feedback.	After each review and after deployment	Continuous Improvement	Steven & Kyle	Feedback check	Updated and improved system

SDLC mapping guidance

Waterfall and V-Model usually emphasize planned phases and formal transitions. Prototyping, Iterative, Incremental, Spiral, and RAD repeat or accelerate selected activities. Scrum normally organizes work into sprints; Kanban uses continuous flow and work-in-progress control; XP emphasizes short feedback cycles and engineering practices. Your mapping must reflect the actual method selected by the team.

9. Part E - Semester Project Milestone Plan

Translate the selected methodology into a realistic semester schedule. Every milestone should have a measurable output and a review, approval, demonstration, test, or feedback point. Use your actual course deadlines when they are already known.

Week(s) / Date	Phase / Sprint / Iteration / Milestone	Planned Features / Scope	Key Activities	Evidence / Deliverable	Review / Approval / Feedback
Week 1 September 19–25, 2026	Sprint 1 – Planning & Requirements	Project scope, requirements, users, and major system features	Finalize requirements, project backlog, and development priorities	Requirements document and prioritized backlog	Instructor/adviser review and feedback
Week 2 September 26–October 2, 2026	Sprint 2 – System Design	Flutter Web interface, REST API structure, PostgreSQL mapping, and architecture	Finalize UI flow, API operations, database mapping, and system design	System architecture, UI plan, API plan, and database mapping	Design review and feedback
Weeks 3–4 October 3–9, 2026	Sprint 3 – Core Features	Login, household registration, and household assessment	Develop and test the initial Flutter Web features and REST API operations	Working login, registration, and assessment features	Feature demonstration and feedback
Weeks 5–6 October 10–16, 2026	Sprint 4 – Vulnerability Classification	ML-based household vulnerability classification and vulnerability results	Integrate assessment data with the ML Classification Service and test classification results	Working vulnerability classification feature	ML feature demonstration, testing, and feedback
Weeks 7–8 October 17–23, 2026	Sprint 5 – Aid Management	Beneficiary list and aid distribution	Develop beneficiary management and aid distribution features and test API/database integration	Working beneficiary list and aid distribution features	Feature review and feedback
Weeks 9–10 October 24–30, 2026	Sprint 6 – Reports & Integration	Reports and complete system integration	Integrate Flutter Web, FastAPI, PostgreSQL, and ML service; conduct system testing	Integrated system and test results	System demonstration and adviser feedback
Final Weeks October 31–November 6, 2026	Final Sprint – Testing & Deployment	Complete system, documentation, and deployment preparation	Fix remaining errors, finalize documentation, and prepare the system for deployment	Final tested system, documentation, and deployment-ready build	Final review, approval, and feedback

Milestone planning rule
The methodology plan must be visible in the schedule. A sprint-based plan should show sprint goals and reviews; an incremental plan should show usable feature increments; a predictive plan should show phase gates and approval points; a Kanban-style plan should show how work enters, moves through, and exits the workflow. Do not use vague milestones such as "continue coding" without an observable deliverable.

10. Part F - DevOps and CI/CD Complementary Delivery Plan

Planning Item	What Must Be Decided	Your Project Decision / Evidence
Version-control workflow	How branches, commits, merges, and code reviews support the selected methodology.	Git and GitHub will be used to track project changes and maintain the project repository.
Continuous integration	When code is integrated and what build/check should run before changes are accepted.	Integrate completed sprint features and check before merging.
Testing practice	What unit, API, database, UI, or acceptance checks will be performed and when.	Test Flutter UI, FastAPI API, PostgreSQL operations, and ML integration during each sprint.
Database change control	How PostgreSQL schema changes, migrations, and seed data will be coordinated with application development.	Coordinate database changes with backend features and keep scripts in database/.
Deployment stages	Local, test/staging, and final deployment targets; identify any approval before release.	Test locally first, then staging when available, before final release.
Environment configuration	How API URLs, credentials, and other environment-specific values are kept outside source code.	Keep API URLs and credentials in environment configuration; no real secrets in GitHub.
Monitoring / feedback	How errors, defects, stakeholder feedback, and completed work are recorded and fed into the next cycle or milestone.	Record errors and feedback as tasks for the next sprint.
Release / rollback	How the team will identify a stable release and recover from a failed deployment or defective change.	Use a tested stable Git version; return to the last stable version if needed.

Recommended minimum methodology evidence in the project repository or shared workspace (adapt as needed):

```
docs/methodology/
  development-methodology-plan.pdf
  methodology-selection-justification.md
```

```
  milestone-plan/
  sprint-or-iteration-notes/
  review-and-feedback-records/
  test-evidence/
  release-notes/
README.md - brief description of the team's development workflow
```

11. Part G - Construct the Development Methodology Plan

Create one clear project-specific methodology plan that explains how your team will manage the semester project from requirements through delivery. The plan may be presented as a structured one-page or multi-page visual, timeline, workflow, or combination of diagram and schedule, but it must remain consistent with the completed worksheets.

No.	Required Element	What Must Be Shown
1	Project identity	Official system title, project problem, major users/stakeholders, and team members.
2	Selected primary methodology	Exact methodology/approach selected and whether it is predictive, iterative, incremental, adaptive, or hybrid.
3	Selection rationale	Project evidence from requirement stability, risk, scope, schedule, team, and stakeholder involvement.
4	SDLC mapping	How planning, requirements, design, development, testing, deployment, and review are organized.
5	Iteration / sprint / flow structure	Length or sequence of iterations/sprints/stages, or the workflow used if the method does not use fixed sprints.
6	Requirements / backlog control	How features, tasks, priorities, changes, and acceptance criteria will be recorded and updated.
7	Team responsibilities	Who plans, develops, tests, reviews, documents, coordinates, or approves work.
8	Semester milestones	Major target dates or weeks, expected deliverables, demonstrations, and completion criteria.
9	Stakeholder involvement	When users, instructor, adviser, client, or other stakeholders provide feedback or approval.
10	Quality and testing	When testing occurs and how defects, rework, verification, or acceptance are handled.
11	DevOps / CI/CD support	Git/GitHub workflow, integration, build/test, database change control, deployment, and feedback practices.
12	Adaptation / control rule	How the team responds when requirements, risks, schedule, or technical conditions change.

Minimum Plan Rule

Your plan must show a complete management path from project requirements or backlog -> planned phase/iteration/sprint/work queue -> development and testing -> review/feedback -> milestone or release. If the selected method is iterative or adaptive, show how the cycle repeats. If it is predictive, show phase gates. If it is hybrid, label exactly where each approach is used.

12. Reference Example - Do Not Copy as Your Final Output

The example below illustrates the decision-and-planning logic expected in the final output. Replace the generic labels with the actual constraints, methodology, iteration/stage structure, milestones, stakeholder reviews, and delivery practices of your own project.

Reference Methodology Selection and Semester Planning Flow

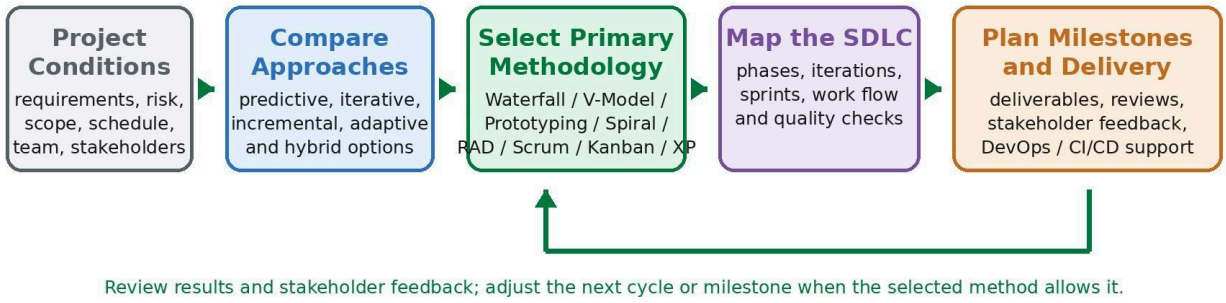
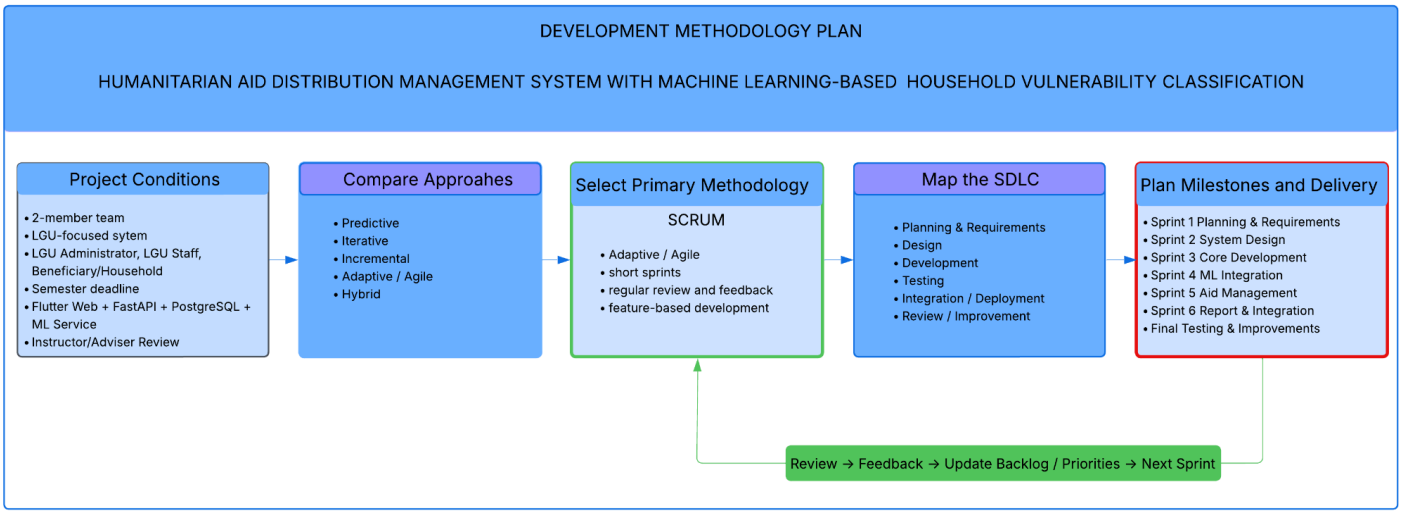


Figure 1. Reference methodology selection and semester planning flow.

Step	Reference Flow
1	Identify project conditions: requirement stability, risk, scope, schedule, team, and stakeholder involvement.
2	Compare predictive, iterative, incremental, adaptive, and hybrid options using the actual project evidence.
3	Select one primary methodology and identify any complementary practices such as prototyping, DevOps, or CI/CD.
4	Map SDLC phases to stages, iterations, sprints, or continuous workflow according to the selected methodology.
5	Define semester milestones, deliverables, testing points, demonstrations, reviews, and stakeholder feedback.
6	Use results and feedback to continue the planned next stage or adapt the next cycle when the selected methodology permits it.



13. Part H - Methodology-to-Project Trace

Selected feature / work package:
Household Assessment and Vulnerability Classification

Target milestone / release: **Sprint 4 - ML Integration**

Step / Iteration	Planned Work	Methodology Principle / Practice	Evidence / Deliverable	Review / Feedback
Sprint 1- Planning	Define household assessment and vulnerability classification requirements	Sprint Planning	Requirements / prioritized backlog	Instructor/adviser feedback
Sprint 2 - Design	Design assessment screen, API flow, database mapping, and ML flow	Sprint Planning	UI, API, database, and ML design	Design review and feedback
Sprint 3 - Development	Develop household assessment and save assessment data	Incremental development	Working assessment feature	Feature testing and review
Sprint 3 - Development	Save assessment data in PostgreSQL	Incremental development	Assessment record stored	Database testing
Sprint 4 - ML Integration	Connect assessment data to ML Classification Service	Integration and testing	Working ML classification connection	ML integration review
Sprint 4 - ML Integration	Display High / Medium / Low vulnerability result	Working increment	Vulnerability result displayed	Result testing and feedback
Sprint 5 - Aid Management	Use vulnerability classification in beneficiary and aid processes	Incremental development	Working aid management feature	Feature review
Sprint 6 - Reports & Integration	Integrate assessment, classification, aid, and reports	System integration	Integrated system	System demonstration and feedback
Final Sprint - Testing & Improvements	Verify assessment and classification in the integrated system	Final testing and improvement	Tested feature / final documentation	Final review

14. Part I - Methodology-Selection Justification

Write 120-180 words explaining why the selected development methodology is appropriate for your own semester project. This will also serve as the basis of the online methodology-selection justification. Address all of the following:

- Name the selected primary methodology and classify it as predictive, iterative, incremental, adaptive, or hybrid as appropriate.
- Use project evidence related to requirement stability, risk, scope, schedule, team capability, and stakeholder involvement.
- Explain why the selected methodology is more appropriate than at least two reasonable alternatives.
- Describe how SDLC phases, iterations, sprints, work flow, and semester milestones will be organized.
- Explain how DevOps and CI/CD practices will complement development and delivery, including one limitation or risk the team must manage.

The selected primary methodology is Scrum, classified as Adaptive / Agile. It is appropriate for our semester project because the main requirements and features are already identified, but some details may change through testing and instructor or adviser feedback. The project also has technical risks because it involves Flutter Web, REST API, PostgreSQL, and a Machine Learning Classification Service. Compared with Waterfall, Scrum is more flexible when changes are needed, while compared with Spiral, Scrum is simpler and more practical for our two-member team and semester schedule. The SDLC will be organized into planning, requirements, design, development, testing, integration, deployment, and improvement. These activities will be completed through short sprints, from Planning and Requirements to System Design, Core Development, ML Integration, Aid Management, Reports and Integration, and final Testing and Improvements. Git/GitHub, CI/CD, testing, and controlled database changes will support development and delivery. One risk is poor coordination or integration issues, which must be managed through regular testing and review.

15. Final Quality Checklist

Checklist A	Checklist B
[ ] The official project title, users, features, and real project conditions are used.	[ ] The selected primary methodology is clearly named and correctly classified.
[ ] Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP were compared.	[ ] Selection criteria use actual evidence about requirements, risk, scope, schedule, team, and stakeholders.
[ ] Scenario analysis includes a methodology choice, reasoning, tradeoff, and alternative.	[ ] The team explains why at least two reasonable alternative methodologies were not selected.
[ ] SDLC phases are mapped consistently to the selected methodology.	[ ] Iterations, sprints, stages, or continuous workflow are represented accurately.
[ ] The semester milestone plan includes dates/weeks, deliverables, and review or feedback points.	[ ] Stakeholder involvement and change-management points are visible in the plan.
[ ] DevOps and CI/CD are treated as complementary delivery practices, not as the primary methodology.	[ ] The plan is feasible for the team size, available time, technical risks, and semester deadline.
[ ] The written methodology-selection justification agrees with all worksheets and the final plan.	[ ] Every group member can explain the methodology, milestone sequence, and major tradeoffs without reading a script.

16. Submission Requirements

- PDF of the final Development Methodology Plan in a clear, readable format.
- Completed Parts A-H and the 120-180 word methodology-selection justification.
- Export or screenshot of the semester milestone timeline, sprint/iteration plan, or task board used to support the final plan.
- Updated project GitHub repository or shared project workspace containing the methodology artifacts under a folder

such as docs/methodology/.

- Repository link and required files submitted through MasaoLMS using the naming convention IT110_LA4_SystemName.
- Deliver a 5-7 minute group presentation using the final methodology plan as the main visual, followed by 2-3 minutes of questions. Every member must participate meaningfully.

Academic Integrity and Competency Validation

The output must represent the team's own project conditions and planning decisions. AI tools may be used to clarify methodology concepts or review ideas, but the final comparison, selection criteria, methodology choice, SDLC mapping, milestone plan, and written justification must be checked, understood, and defended by the team. The instructor may ask any member to explain why a methodology was selected or trace how one feature will move through the planned process without assistance.

17. Scoring Rubric - 80 Points

Partial credit may be awarded when evidence is present but incomplete, inconsistent, or technically inaccurate. For the presentation criterion, points may be reduced for unclear reasoning, unequal participation, excessive reading, inconsistent milestone mapping, or inability to defend the selected methodology against reasonable alternatives.

Criterion	Evidence for Full Credit	Points
Methodology comparison accuracy	The comparison correctly distinguishes the major characteristics, strengths, limitations, and appropriate use of Waterfall, V-Model, Prototyping, Iterative, Incremental, Spiral, RAD, Scrum, Kanban, and XP.	10
Scenario analysis and tradeoff reasoning	Each scenario uses relevant project conditions to select a defensible method, explains why it fits, identifies a limitation, and considers a reasonable alternative or hybrid option.	8
Project-specific selection criteria	Requirement stability, risk, scope, schedule, team capability, stakeholder involvement, and need for change/feedback are evaluated using evidence from the actual project.	10
Selected methodology and written justification	The primary methodology is clearly named, correctly classified, supported by project evidence, compared with alternatives, and justified in a coherent 120-180 word explanation.	12
SDLC, iteration, sprint, or flow mapping	Planning, requirements, design, development, testing, deployment, review, and improvement are mapped accurately to the selected method and its stages, cycles, sprints, or continuous workflow.	12
Semester milestone plan	The schedule is realistic, project-specific, and aligned with the selected methodology; milestones contain measurable scope, deliverables, review/feedback points, and completion evidence.	12
DevOps and CI/CD complementary delivery plan	Version control, integration, testing, database change control, deployment, configuration, monitoring/feedback, and release/rollback practices appropriately support the selected methodology.	6
Group presentation and oral defense	The presentation is clear, organized, and within time; all members participate and correctly defend the methodology choice, project mapping, milestones, tradeoffs, and delivery practices.	10

Raw Score	Equivalent	Checked by	Date
/ 80	_____	_____	_____

18. Post-Laboratory Reflection

Which project selection criterion had the greatest influence on your methodology choice, and why?

Technical / project risk had the greatest influence because our project uses REST API, PostgreSQL, and a Machine Learning Classification Service. These components may have integration issues during development, so Scrum allows us to test and adjust the system through short sprints and regular feedback.

Which reasonable alternative methodology did your group reject, and what project condition made it less suitable?

We rejected Waterfall because some project details may change during development based on testing and instructor/adviser feedback. Waterfall is less flexible when changes happen after a phase has been completed.

Where will the most important iteration, sprint review, phase gate, or workflow review occur in your semester plan?

The most important review will occur during the Sprint 6 – Reports & Integration, where the complete system will be integrated and demonstrated. The feedback from this review will guide the Final Sprint Testing & Improvements.

How will stakeholder feedback or requirement changes be handled without losing control of scope and schedule?

Feedback and requirement changes will be reviewed at the end of each sprint. Accepted changes will be added or reprioritized in the backlog and scheduled for a later sprint, while the team will keep the main semester scope and deadline.

Which DevOps or CI/CD practice should your team establish first, and what evidence will show that it is working?

The team should first establish a Git/GitHub version-control workflow with regular integration and testing. Evidence that it is working will include organized branches, commits, merged changes, and successful build/test checks before changes are accepted.